

Verificación Funcional de un Alineador de Datos mediante UVM en SystemVerilog

Esteban Sánchez Acevedo 2022437779

Escuela de Ingeniería en Electrónica

Instituto Tecnológico de Costa Rica

Cartago, Costa Rica

estsanchez@estudiantec.cr

Esteban Segura García 2022129260

Escuela de Ingeniería en Electrónica

Instituto Tecnológico de Costa Rica

Cartago, Costa Rica

essegura@estudiantec.cr

Resumen—Este trabajo presenta el ambiente de verificación funcional de un módulo alineador de datos (*aligner*) descrito en SystemVerilog y verificado con la metodología universal de verificación (UVM). El objetivo central del diseño del *testbench* fue evitar pruebas codificadas de forma rígida (*hardcoded*): en su lugar se implementó una *única prueba general* cuyo comportamiento se controla por completo mediante argumentos de línea de comando (*plusargs*). Estos argumentos no fijan valores de estímulo, sino que ajustan los pesos de restricciones *dist* ubicadas en los *sequence items*, de modo que el solucionador de restricciones genera aleatoriamente los casos requeridos. Se describen la arquitectura del ambiente, los agentes APB y MD, el *scoreboard* con modelo de registros (RAL) sincronizado por adaptador y por predicción explícita, la metodología de pesos y el plan de pruebas completo con su mapeo a *plusargs*. Los resultados muestran que la totalidad de los escenarios del plan de pruebas se alcanzan sin modificar el código fuente, únicamente variando los *plusargs*.

Index Terms—UVM, SystemVerilog, verificación funcional, restricciones, *plusargs*, APB, RAL, alineador.

I. INTRODUCCIÓN

La verificación funcional consume una fracción mayoritaria del esfuerzo en el diseño de circuitos digitales [1]. La metodología UVM [2] estandariza la construcción de ambientes reutilizables basados en componentes (agentes, *drivers*, *monitors*, *scoreboards*) y en estímulos generados aleatoriamente bajo restricciones.

El presente informe documenta la verificación de un módulo *aligner*, cuya función es recibir bloques de datos con un tamaño y un desplazamiento arbitrarios por un canal de recepción (RX) y reensamblarlos en transferencias alineadas en un canal de transmisión (TX), de acuerdo con una configuración escrita por un bus APB. El módulo además contabiliza las transferencias descartadas y genera interrupciones mediante un conjunto de registros de control y estado.

Un requisito explícito del curso es que *ninguna prueba esté codificada de forma rígida*: no deben existir múltiples clases de prueba con valores fijos para cada caso, sino una sola prueba general cuyos distintos escenarios se obtengan activando restricciones o variando valores mediante *plusargs*. La Sección V detalla cómo se cumple este requisito y la Sección VI mapea cada objetivo de verificación a su combinación de *plusargs*.

Cuadro I
SEÑALES DE LA INTERFAZ APB (*APB_IF*)

Señal	Dir.	Descripción
PADDR	Ent.	Dirección del registro (16 bits).
PWRITE	Ent.	1 = escritura, 0 = lectura.
PSEL	Ent.	Selección del esclavo.
PENABLE	Ent.	Fase de acceso.
PWDATA	Ent.	Bus de datos de escritura (32 bits).
PRDATA	Sal.	Bus de datos de lectura (32 bits).
PREADY	Sal.	El esclavo completó la transacción.
PSLVERR	Sal.	Error del esclavo.

II. DISPOSITIVO BAJO PRUEBA

II-A. Descripción del DUT

El DUT es el módulo *cfs_aligner*, parametrizado con un ancho de datos *ALGN_DATA_WIDTH* = 32 y una profundidad de FIFO *FIFO_DEPTH* = 8. Internamente posee una FIFO de recepción que almacena los bytes válidos de cada transferencia RX y una FIFO de transmisión que entrega los datos ya alineados según la configuración del registro CTRL. Las transferencias que violan la regla de alineamiento (Sección II-D) se descartan e incrementan un contador de *drops*.

II-B. Interfaces y Señales

El DUT se comunica con el ambiente mediante dos interfaces. La interfaz *apb_if* implementa el protocolo de configuración AMBA APB [5] y la interfaz *md_if* transporta los datos en los canales RX y TX. Las Tablas I y II describen sus señales. Ambas interfaces definen *clocking blocks* separados para el *driver* (vista activa) y el *monitor* (vista pasiva), garantizando muestreo sincrónico.

II-C. Mapa de registros

La Tabla III resume el mapa de registros, obtenido del modelo RAL autogenerado con PeakRDL.

Los cinco bits de *IRQEN/IRQ* corresponden, de menor a mayor peso, a: *RX_FIFO_EMPTY*, *RX_FIFO_FULL*, *TX_FIFO_EMPTY*, *TX_FIFO_FULL* y *MAX_DROP*. El bit *MAX_DROP* se activa únicamente cuando el contador *CNT_DROP* satura en 255.

Cuadro II
SEÑALES DE LA INTERFAZ MD (MD_IF)

Señal	Dir.	Descripción
md_rx_valid	Ent.	Transferencia RX válida.
md_rx_data	Ent.	Datos RX (32 bits).
md_rx_offset	Ent.	Byte inicial (0–3).
md_rx_size	Ent.	Cantidad de bytes.
md_rx_ready	Sal.	El DUT acepta la transferencia RX.
md_rx_err	Sal.	Transferencia RX ilegal (descartada).
md_tx_valid	Sal.	Transferencia TX válida.
md_tx_data	Sal.	Datos TX alineados (32 bits).
md_tx_offset	Sal.	Offset configurado.
md_tx_size	Sal.	Size configurado.
md_tx_ready	Ent.	El consumidor acepta la TX.
md_tx_err	Ent.	Error inyectado por el consumidor.

Cuadro III
MAPA DE REGISTROS DEL DUT

Registro	Dir.	Campos relevantes
CTRL	0x00	SIZE[2:0], OFFSET[9:8], CLR[16]
STATUS	0x0C	CNT_DROP[7:0], RX_LVL[11:9], TX_LVL[19:16]
IRQEN	0xF0	[4:0] habilitación de interrupciones
IRQ	0xF4	[4:0] interrupciones pendientes (WIC)

II-D. Regla de alineamiento legal

Una transferencia RX se considera *legal* si cumple

$$\left(\frac{\text{ALGN_DATA_WIDTH}}{8} + \text{offset} \right) \bmod \text{size} = 0, \quad (1)$$

con $\text{size} \in \{1, 2, 4\}$. Para un bus de 32 bits, $\text{ALGN_DATA_WIDTH}/8 = 4$, por lo que la condición se reduce a $(4 + \text{offset}) \bmod \text{size} = 0$. De aquí: $\text{size} = 1$ admite cualquier offset; $\text{size} = 2$ admite $\text{offset} \in \{0, 2\}$; y $\text{size} = 4$ admite únicamente $\text{offset} = 0$. Cualquier combinación fuera de estas reglas (o $\text{size} \in \{0, 3\}$) provoca que el DUT marque error y descarte la transferencia, incrementando CNT_DROP.

III. ARQUITECTURA DEL AMBIENTE DE VERIFICACIÓN

La Figura 1, incluida en el Apéndice A, muestra la estructura del ambiente UVM. El *test* de nivel superior instancia un *environment* (*aligner_env*) que contiene dos agentes (APB y MD), un *scoreboard*, el modelo de registros (RAL) y su adaptador. Un bloque de aserciones (SVA) observa directamente las señales del DUT.

III-A. Conexiones de análisis y del RAL

El monitor APB publica cada transacción observada en un puerto de análisis conectado al *scoreboard*. El monitor MD expone dos puertos independientes (*ap_rx* y *ap_tx*). El *aligner_env* construye el modelo RAL, lo bloquea con *lock_model()* y lo expone por *uvm_config_db*. Durante la fase de conexión, asocia el mapa de registros

al secuenciador APB mediante el adaptador y habilita la predicción automática, como muestra el Listado 1.

Listing 1. Conexiones del RAL y los puertos de análisis en el environment

```
1 // mapa RAL -> secuenciador APB via adaptador
2 reg_model.default_map.set_sequencer(
3     apb_agt.sequencer, reg_adapter);
4 reg_model.default_map.set_auto_predict(1);
5
6 // monitores -> scoreboard
7 apb_agt.monitor.ap.connect(scoreboard.apb_ap);
8 md_agt.ap_rx.connect(scoreboard.md_rx_ap);
9 md_agt.ap_tx.connect(scoreboard.md_tx_ap);
```

IV. DESCRIPCIÓN DE LOS COMPONENTES

IV-A. Sequence items

IV-A1. apb_seq_item: Encapsula una operación APB de alto nivel mediante el campo *op_type* (OP_CTRL_CFG, OP_IRQEN_CFG, OP_STATUS_READ, OP_IRQ_READ, OP_IRQ_CLR, OP_CTRL_RECONFIG). Contiene los campos de bajo nivel (*addr*, *data*, *write*, *rdata*, *slverr*) y los campos aleatorios de alto nivel *ctrl_size*, *ctrl_offset*, *irqen_bits* y *do_irq_clr*, todos gobernados por restricciones *dist* ponderadas. La restricción *c_ctrl_valid_combo* garantiza que solo se generen pares (SIZE, OFFSET) válidos según la Ecuación (1).

IV-A2. md_seq_item: Modela una transferencia del canal MD. Los campos *rx_size*, *rx_offset* e *is_illegal* están controlados por pesos; el dato *rx_data* es completamente aleatorio. El Listado 2 muestra las restricciones que implementan la regla legal/ilegal: cuando se solicita una transferencia ilegal, el solucionador elige una combinación de *size* y *offset* que garantiza el error sin fijar valores a mano.

Listing 2. Restricciones ponderadas en md_seq_item

```
1 constraint c_illegal_weight {
2     is_illegal dist { 1'b0 := (100 - w_illegal),
3                     1'b1 := w_illegal };
4 }
5 constraint c_size_dist {
6     rx_size <= 4;
7     if (!is_illegal)
8         rx_size dist { 3'h1 := w_size1,
9                     3'h2 := w_size2,
10                    3'h4 := w_size4 };
11     else
12         rx_size inside {3'h0, 3'h2, 3'h3, 3'h4};
13 }
14 constraint c_legal_combo {
15     if (!is_illegal)
16         ((4 + rx_offset) % rx_size) == 0;
17     else if (rx_size == 3'h3) rx_offset != 2'h2;
18     else if (rx_size == 3'h2)
19         !(rx_offset inside {2'h0, 2'h2});
20     else if (rx_size == 3'h4) rx_offset != 2'h0;
21 }
```

IV-B. Secuencias

IV-B1. apb_sequence: Es la única secuencia APB del ambiente. Una instancia ejecuta una sola operación indicada por el campo *op*. La secuencia copia los pesos al *item*, invoca *randomize()* y, según la operación, traduce los campos de alto nivel en *addr/data/write* (Listado 3). Tras la ejecución, el campo *last_item* permite al test leer los

valores efectivamente generados. Para OP_IRQ_READ, si el solucionador decide limpiar la interrupción, la secuencia emite automáticamente una segunda transacción de escritura uno-a-cero (W1C).

Listing 3. Cuerpo de la secuencia APB unificada

```

1 virtual task body();
2   apb_seq_item item;
3   item = apb_seq_item::type_id::create(...);
4   item.w_ctrl_size = w_ctrl_size; // copiar pesos
5   item.op_type = op;
6   start_item(item);
7   if (!item.randomize())
8     `uvm_fatal("APB_SEQ", "randomize() fallo")
9   case (op)
10    OP_CTRL_CFG, OP_CTRL_RECONFIG: begin
11      item.addr = ADDR_CTRL;
12      item.data = (32'(item.ctrl_offset) << 8)
13                | 32'(item.ctrl_size);
14      item.write = 1'b1;
15    end
16    // ... IRQEN_CFG, STATUS_READ, IRQ_READ
17  endcase
18  finish_item(item);
19  last_item = item;
20 endtask

```

IV-B2. *md_rx_multiple_seq*: Envía N transferencias por el canal RX. Antes de cada `randomize()` copia los pesos al *item*, de modo que ningún valor de datos, offset o size queda fijado en el código. El número de transferencias y todos los pesos provienen del test.

IV-C. Drivers

IV-C1. *apb_driver*: Traduce las transacciones en señales sobre la interfaz APB siguiendo el protocolo de dos fases: en la fase de *setup* activa PSEL y presenta dirección y datos; en la fase de *access* activa PENABLE y espera a que PREADY sea exactamente 1 antes de capturar PRDATA y PSLVERR.

IV-C2. *md_rx_driver*: Conduce el canal RX. En reposo mantiene `md_rx_valid=0`; al recibir un *item* presenta los datos, espera el *handshake* (`md_rx_ready`) y captura la respuesta de error (Listado 4).

Listing 4. Conducción de una transferencia RX

```

1 task drive_transfer(md_seq_item req);
2   @(vif.rx_driver_cb);
3   vif.rx_driver_cb.md_rx_valid <= 1'b1;
4   vif.rx_driver_cb.md_rx_data <= req.rx_data;
5   vif.rx_driver_cb.md_rx_offset <= req.rx_offset;
6   vif.rx_driver_cb.md_rx_size <= req.rx_size;
7   do @(vif.rx_driver_cb);
8   while (vif.md_rx_ready !== 1'b1);
9   req.rx_err = vif.md_rx_err; // captura error
10  vif.rx_driver_cb.md_rx_valid <= 1'b0;
11 endtask

```

IV-C3. *md_tx_driver*: Actúa como consumidor del canal TX. Mantiene `md_tx_ready` según un peso `ready_weight` configurable en tiempo de ejecución, lo que permite inyectar contrapresión (*back-pressure*). El Listado 5 muestra cómo el peso determina la disponibilidad de forma probabilística.

Listing 5. Contrapresión ponderada en el driver TX

```

1 local function void update_ready_value();
2   if (ready_weight == 100) ready_value = 1'b1;
3   else if (ready_weight == 0) ready_value = 1'b0;
4   else ready_value =

```

```

5   ($urandom_range(0,99) < ready_weight);
6 endfunction

```

IV-D. Monitores

IV-D1. *apb_monitor*: Detecta el inicio de una transacción cuando PSEL y PENABLE están activos, espera PREADY, reconstruye el *item* (incluyendo *rdata* en las lecturas) y lo publica por su puerto de análisis.

IV-D2. *md_monitor*: Ejecuta dos hilos paralelos, uno por canal. Cada hilo detecta el *handshake* valid && ready y publica la transferencia observada en el puerto correspondiente (`ap_rx` o `ap_tx`), como ilustra el Listado 6.

Listing 6. Captura del handshake en el monitor MD

```

1 if (vif.monitor_cb.md_rx_valid === 1'b1 &&
2     vif.monitor_cb.md_rx_ready === 1'b1) begin
3   trans = md_seq_item::type_id::create("rx_trans");
4   trans.is_rx = 1'b1;
5   trans.rx_data = vif.monitor_cb.md_rx_data;
6   trans.rx_offset = vif.monitor_cb.md_rx_offset;
7   trans.rx_size = vif.monitor_cb.md_rx_size;
8   trans.rx_err = vif.monitor_cb.md_rx_err;
9   ap_rx.write(trans);
10 end

```

IV-E. Agentes

El *apb_agent* (activo) instancia *monitor*, *driver* y secuenciador, y conecta el puerto del secuenciador al *driver*. El *md_agent* instancia el monitor, los dos *drivers* (RX y TX) y un secuenciador, y reexpone los puertos de análisis del monitor hacia el ambiente.

IV-F. Adaptador y RAL

El *apb_adapter* (Listado 7) implementa `reg2bus` y `bus2reg`, traduciendo entre las operaciones de registro de UVM y el *apb_seq_item*. Junto con `set_auto_predict(1)`, mantiene actualizado el valor reflejado del RAL.

Listing 7. Adaptador de registros APB

```

1 function uvm_sequence_item reg2bus(
2   const ref uvm_reg_bus_op rw);
3   apb_seq_item apb_op =
4     apb_seq_item::type_id::create("apb_op");
5   apb_op.addr = rw.addr;
6   apb_op.data = rw.data;
7   apb_op.write = (rw.kind == UVM_WRITE);
8   return apb_op;
9 endfunction

```

IV-G. Scoreboard

Recibe tres flujos de análisis (APB, RX, TX) y verifica: (i) que las transferencias RX legales no produzcan error y las ilegales sí; (ii) que cada transferencia TX respete el SIZE y OFFSET configurados; y (iii) que los bytes reensamblados en TX coincidan con los bytes válidos recibidos por RX, almacenados en una cola interna. Para conocer la configuración vigente, el *scoreboard* sincroniza el RAL mediante `predict()` en cada escritura observada a CTRL y lo consulta con `get()` al verificar las TX (Listado 8).

Listing 8. Sincronización del RAL en el scoreboard

```

1 if (trans.write && (aligned_addr == CTRL_ADDR)
2   && !trans.silverr)
3   void' (reg_model.CTRL.predict(trans.data));
4 // ...
5 ctrl_size_val = reg_model.CTRL.SIZE.get();
6 ctrl_offset_val = reg_model.CTRL.OFFSET.get();

```

IV-H. Environment y Test

El aligner_env crea y conecta todos los componentes (Listado 1). El aligner_test lee los plusargs en su run_phase, fija valores por defecto razonables y ejecuta siempre el mismo esqueleto. El aligner_tb_top genera el reloj y el reset, instancia el DUT y las interfaces, las publica por uvm_config_db y lanza la prueba con run_test(.aligner_test").

V. METODOLOGÍA: RESTRICCIONES PONDERADAS Y PLUSARGS

La idea central del ambiente es que el estímulo nunca se fija manualmente. Cada campo aleatorio del *sequence item* está gobernado por una restricción dist cuyos pesos son variables configurables. Un peso de 0 desactiva un valor, un peso de 100 lo fuerza, y los valores intermedios producen una distribución ponderada. De esta forma, “activar una restricción” y “cambiar un valor” son la misma acción: ajustar un peso.

Existe una sola clase de prueba (aligner_test). Los distintos escenarios del plan de pruebas se obtienen variando los plusargs en la línea de comando, nunca editando el código. El test lee todos los plusargs y los propaga a las secuencias, que a su vez los copian a los *items* antes de randomizar (Listado 9).

Listing 9. Lectura de plusargs en el test general

```

1 void' ($value$plusargs("NUM_TRANSFERS=%d", num_transfers));
2 void' ($value$plusargs("W_SIZE1=%d", w_size1));
3 void' ($value$plusargs("W_ILLEGAL=%d", w_illegal));
4 void' ($value$plusargs("W_IRQEN_MAX_DROP=%d",
5   w_irqen_max_drop));
6 void' ($value$plusargs("N_RECONFIG=%d", n_reconfig));
7 // ... resto de plusargs

```

VI. PLAN DE PRUEBAS

La Tabla V lista los plusargs disponibles y su efecto. La Tabla VI, incluida en el Apéndice A, mapea cada objetivo de verificación a la combinación de plusargs que lo alcanza, todos ejecutados sobre la misma prueba aligner_test.

VII. CASOS DE ESQUINA

Los casos de esquina (*corner cases*) no constituyen pruebas separadas: son condiciones límite que la prueba general aligner_test puede alcanzar configurando los pesos adecuados. Esta sección describe los casos considerados, agrupados según el mecanismo del DUT que estresan, y la Tabla VII, incluida en el Apéndice A, los resume junto con la combinación de plusargs que los provoca.

Cuadro IV
ARGUMENTOS DE LÍNEA DE COMANDO (PLUSARGS)

Plusarg	Efecto
NUM_TRANSFERS	Cantidad de transferencias RX.
W_SIZE1/2/4	Pesos del tamaño RX (1, 2, 4 bytes).
W_OFF0..3	Pesos del offset RX (0–3).
W_ILLEGAL	Peso de transferencias ilegales (0–100).
TX_READY_WEIGHT	Disponibilidad del consumidor TX.
W_CTRL_SIZE1/2/4	Pesos del SIZE configurado en CTRL.
W_CTRL_OFF0..3	Pesos del OFFSET configurado en CTRL.
W_IRQEN_*	Pesos por bit de IRQEN.
W_IRQ_CLR	Peso de limpieza WIC tras leer IRQ.
N_RECONFIG	Número de reconfiguraciones de CTRL.

VII-A. Casos límite de la regla de alineamiento

La regla $(4 + \text{offset}) \bmod \text{size} = 0$ (Ecuación (1)) define una frontera estrecha entre transferencias legales e ilegales. Se consideraron los extremos de esa frontera:

- Palabra completa (máximo tamaño): $\text{size}=4$ solo es legal con $\text{offset}=0$; es el único par válido para el tamaño máximo y el que produce una TX de cuatro bytes.
- Offset máximo: $\text{offset}=3$ solamente es legal cuando $\text{size}=1$; cualquier otro tamaño con ese offset es ilegal.
- Tamaño nulo ($\text{size}=0$): siempre ilegal, pues la regla implica una división por cero. El *item* lo incluye en el *pool* ilegal sin restringir el offset.
- Tamaño no potencia de dos ($\text{size}=3$): ilegal por construcción; el constraint excluye $\text{offset}=2$ del *pool* ilegal porque $(4 + 2) \bmod 3 = 0$ se colaría como legal.
- Desalineamientos de borde: $\text{size}=2$ con $\text{offset} \in \{1, 3\}$ y $\text{size}=4$ con $\text{offset} \neq 0$, que son las combinaciones inválidas inmediatamente adyacentes a las legales.

VII-B. Casos límite del contador de drops e interrupción

El contador CNT_DROP es de ocho bits y la interrupción MAX_DROP se activa únicamente cuando satura en 255. Se verificó el comportamiento en la frontera exacta:

- Justo por debajo del umbral: 254 transferencias ilegales mantienen $\text{IRQ}[4]=0$.
- En el umbral exacto: 255 *drops* disparan MAX_DROP y dejan $\text{CNT_DROP}=0 \times \text{FF}$.
- Saturación sostenida: con más de 255 *drops* el contador permanece en $0 \times \text{FF}$ sin desbordarse.

Estos tres puntos se alcanzan con $+W_ILLEGAL=100$ y variando NUM_TRANSFERS entre 254, 255 y 300.

VII-C. Casos límite de las FIFOs

Con $\text{FIFO_DEPTH}=8$, los niveles de las FIFOs son condiciones de borde relevantes:

- FIFO TX llena: con $+TX_READY_WEIGHT=0$ el consumidor nunca acepta datos, por lo que la FIFO de transmisión se llena y se observa TX_LVL en su máximo (condición TX_FIFO_FULL).
- FIFO TX vacía: con $+TX_READY_WEIGHT=100$ el DUT drena inmediatamente y TX_LVL termina en cero (condición TX_FIFO_EMPTY).

- Byte residual: cuando los bytes válidos acumulados no completan un paquete del `SIZE` configurado, el remanente permanece en la FIFO de transmisión; el *scoreboard* lo contempla y `TX_LVL` refleja ese residuo al final de la corrida.

VII-D. Casos límite de configuración

- Reconfiguración en caliente: con `+N_RECONFIG=k` el registro `CTRL` cambia de `SIZE/OFFSET` entre bloques de tráfico, verificando que las TX posteriores respeten la configuración vigente (el *scoreboard* se resincroniza mediante `predict()`).
- Limpieza de interrupción (WIC): con `+W_IRQ_CLR=100` junto a `MAX_DROP`, tras leer `IRQ` la secuencia emite la escritura uno-a-cero y se comprueba que el bit se limpia.
- Tráfico totalmente legal o totalmente ilegal: `+W_ILLEGAL=0` (ningún *drop*) y `+W_ILLEGAL=100` (ninguna TX producida) son los dos extremos del espacio de estímulo RX.

VIII. FLUJO DE EJECUCIÓN

El procesamiento de una corrida sigue la siguiente secuencia:

1. El `tb_top` genera reloj y *reset*, publica las interfaces por `uvm_config_db` y lanza `aligner_test`.
2. El test lee los plusargs y configura `CTRL` (y opcionalmente `IRQEN`) mediante la `apb_sequence`.
3. La `md_rx_multiple_seq` envía las transferencias RX en uno o varios bloques; entre bloques el test reconfigura `CTRL` si `N_RECONFIG > 0`.
4. Los monitores publican cada transacción; el *scoreboard* compara RX contra TX y verifica el reensamblado byte a byte.
5. El test lee `STATUS` (y `IRQ` si hay interrupciones habilitadas) y finaliza bajando la objeción.

IX. RESULTADOS

Todos los escenarios de la Tabla VI se ejecutaron sin modificar el código fuente. A modo de ejemplo, el escenario 4 (`+W_ILLEGAL=0 +NUM_TRANSFERS=100`) con una configuración aleatoria `SIZE=2`, `OFFSET=2` produjo el resumen:

```
PASSED=380   FAILED=0
```

El conteo se descompone en 100 verificaciones de transferencias RX legales y 70 paquetes TX, cada uno con cuatro verificaciones (offset, size y dos bytes de datos), confirmando el reensamblado correcto. El byte residual que no alcanza a formar un paquete completo permanece en la FIFO de transmisión, consistente con `TX_LVL` observado.

Para el escenario 8 se verificó que con 300 transferencias ilegales el contador `CNT_DROP` satura en `0xFF` y la interrupción `MAX_DROP` (`IRQ[4]`) se activa, mientras que con 50 transferencias (escenario 9) la interrupción permanece inactiva, consistente con la especificación del registro `IRQ`.

X. COBERTURA

[Sección en desarrollo por el integrante a cargo de cobertura.]

XI. CONCLUSIONES

Se construyó un ambiente de verificación UVM completo para un módulo alineador de datos cumpliendo el requisito de no tener pruebas codificadas de forma rígida. La estrategia de restricciones ponderadas ubicadas en los *sequence items*, combinada con una única prueba general parametrizada por plusargs, permitió alcanzar todos los escenarios del plan de pruebas variando únicamente los argumentos de invocación. El estímulo concreto siempre es producto del solucionador de restricciones, lo que preserva la aleatoriedad y la reutilización del ambiente. La verificación de datos mediante el *scoreboard* con sincronización del RAL por adaptador y por `predict()` confirmó el reensamblado correcto y el comportamiento esperado de las interrupciones.

APÉNDICE A

FIGURAS Y CUADROS COMPLEMENTARIOS

En este apéndice se agrupan el diagrama de arquitectura (Figura 1) y los cuadros de mayor extensión: el plan de pruebas (Tabla VI) y los casos de esquina (Tabla VII).

REFERENCIAS

- [1] C. Spear, *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*, 2nd ed. New York, NY: Springer, 2008.
- [2] Accellera, *Universal Verification Methodology (UVM) 1.2 User's Guide*, 2015.
- [3] IEEE, *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017.
- [4] IEEE, *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2-2020.
- [5] ARM, *AMBA APB Protocol Specification*, ARM IHI 0024.

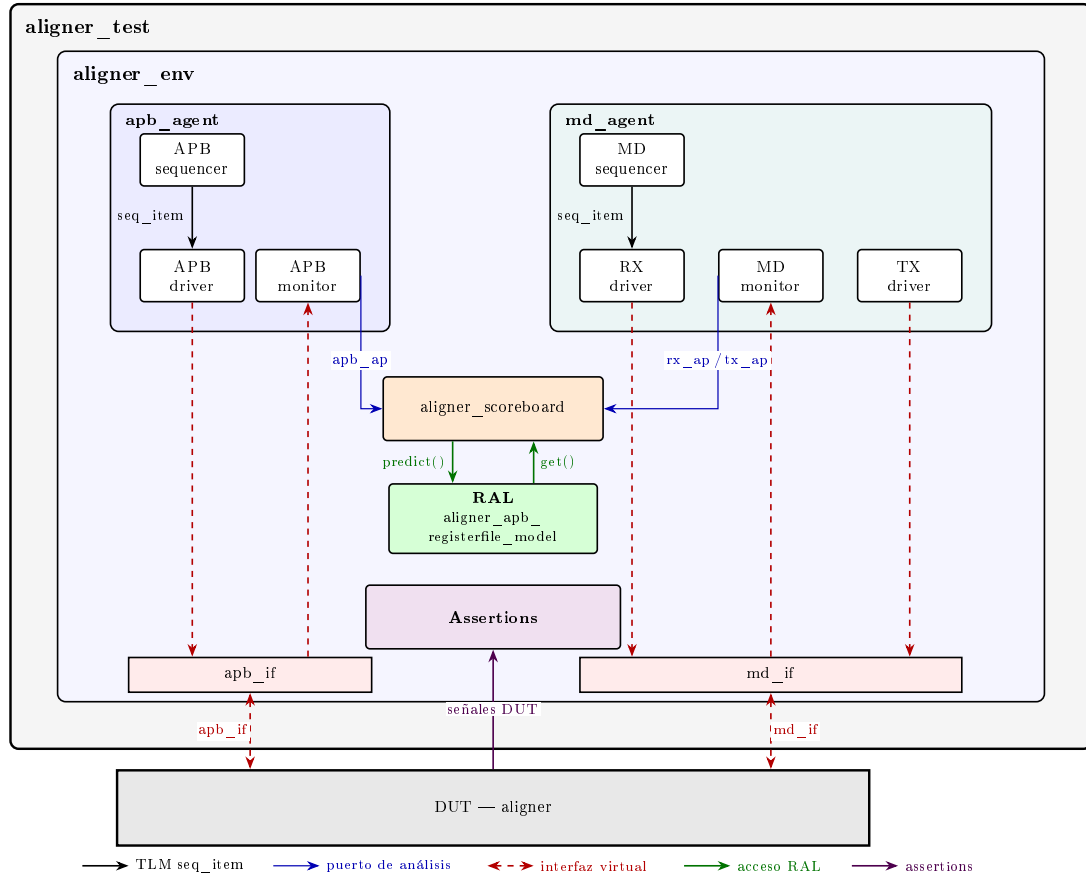


Figura 1. Arquitectura del ambiente de verificación UVM. Los *drivers* conducen el DUT a través de las interfaces virtuales; los *monitors* publican las transacciones observadas hacia el *scoreboard* por puertos de análisis. El RAL se conecta al secuenciador APB mediante el adaptador y se sincroniza con el *scoreboard* por `predict()`/`get()`.

Cuadro V
PLAN DE PRUEBAS Y SU MAPEO A PLUSARGS SOBRE LA PRUEBA GENERAL ALIGNER_TEST

#	Objetivo de verificación	Plusargs	Resultado esperado
1	Configurar CTRL solo en bytes (size=1)	+W_CTRL_SIZE1=100 +W_CTRL_SIZE2=0 +W_CTRL_SIZE4=0	TX con size=1, cualquier offset.
2	Configurar CTRL solo en words (size=4)	+W_CTRL_SIZE4=100 +W_CTRL_SIZE1=0 +W_CTRL_SIZE2=0	TX con size=4, offset=0.
3	Solo offset base (0)	+W_CTRL_OFF0=100 +W_CTRL_OFF1=0 +W_CTRL_OFF2=0 +W_CTRL_OFF3=0	OFFSET=0 en todas las TX.
4	Tráfico RX solo legal	+W_ILLEGAL=0 +NUM_TRANSFERS=100	CNT_DROP=0, sin errores.
5	Mezcla legal/ilegal	+W_ILLEGAL=30 +NUM_TRANSFERS=100	RX ilegales con rx_err=1.
6	Tráfico totalmente ilegal	+W_ILLEGAL=100 +NUM_TRANSFERS=100	Todas las RX descartadas.
7	Contrapresión del consumidor TX	+TX_READY_WEIGHT=20 +NUM_TRANSFERS=100	Acumulación en FIFO TX.
8	Interrupción MAX_DROP al saturar	+W_IRQEN_MAX_DROP=100 +W_ILLEGAL=100 +NUM_TRANSFERS=300	IRQ[4]=1, CNT_DROP=255.
9	MAX_DROP no dispara con < 255 drops	+W_IRQEN_MAX_DROP=100 +W_ILLEGAL=100 +NUM_TRANSFERS=50	IRQ=0x00.
10	Limpieza de interrupción (WIC)	+W_IRQEN_MAX_DROP=100 +W_IRQ_CLR=100 +W_ILLEGAL=100 +NUM_TRANSFERS=300	IRQ se limpia tras lectura.
11	Reconfiguración dinámica de CTRL	+N_RECONFIG=3 +NUM_TRANSFERS=200	4 bloques RX con CTRL distinto.
12	Estrés combinado	+NUM_TRANSFERS=500 +W_ILLEGAL=30 +TX_READY_WEIGHT=50 +N_RECONFIG=3	Sin errores de scoreboard.

Cuadro VI
CASOS DE ESQUINA CONSIDERADOS Y SU FORMA DE ALCANCE SOBRE LA PRUEBA GENERAL

#	Caso de esquina	Cómo se alcanza (plusargs / mecanismo)	Comportamiento esperado
1	Palabra completa (tamaño máximo)	+W_CTRL_SIZE4=100 +W_CTRL_OFF0=100	TX de 4 bytes alineada, sin <i>drops</i> .
2	Offset máximo legal	+W_SIZE1=100 +W_OFF3=100	RX legal con rx_err=0.
3	Halfword en offset par alto	+W_SIZE2=100 +W_OFF2=100	RX legal (size=2, offset=2).
4	Tamaño nulo (size=0)	+W_ILLEGAL=100 (pool ilegal)	rx_err=1, transferencia descartada.
5	Tamaño no potencia de 2 (size=3)	+W_ILLEGAL=100 (pool ilegal)	rx_err=1, descartada.
6	Halfword/word desalineados	+W_ILLEGAL=100 (pool ilegal)	rx_err=1 en size=2 off impar y size=4 off≠0.
7	CNT_DROP bajo el umbral	+W_ILLEGAL=100 +NUM_TRANSFERS=254	IRQ[4]=0, CNT_DROP=254.
8	CNT_DROP en el umbral exacto	+W_ILLEGAL=100 +NUM_TRANSFERS=255	IRQ[4]=1, CNT_DROP=0xFF.
9	CNT_DROP saturado	+W_ILLEGAL=100 +NUM_TRANSFERS=300	CNT_DROP se mantiene en 0xFF.
10	FIFO TX llena (back-pressure total)	+TX_READY_WEIGHT=0 +NUM_TRANSFERS=100	TX_LVL al máximo, TX_FIFO_FULL.
11	FIFO TX drenada por completo	+TX_READY_WEIGHT=100	TX_LVL=0 al final.
12	Byte residual sin paquete completo	Bytes acumulados no múltiplos del SIZE	Remanente en FIFO, TX_LVL>0.
13	Reconfiguración en caliente de CTRL	+N_RECONFIG=3	TX respetan la config vigente en cada bloque.
14	Limpieza de interrupción (WIC)	+W_IRQEN_MAX_DROP=100 +W_IRQ_CLR=100	IRQ se limpia tras la lectura.